act as end-to-end tests, verifying the flow of information through the whole system. If the application is designed well, the business logic will be localized in the form of a well-defined service API that can be tested effectively using non-web tests.

If, on the other hand, the application wasn't designed with a clean service layer, business logic may be scattered through several layers of the system, and there may be no easy way to isolate particular operations. This is often the case for legacy applications, or it may be the result of poor architectural choices; in either case, pragmatic teams may use web testing to verify both the UI behavior and the business logic.

It's increasingly rare these days to see an application without any client-side behavior. Modern applications need to provide rich, interactive experiences that can't be achieved with older approaches. This can only be done by including at least some business logic and behavior within the UI layer. Modern JavaScript-based frameworks such as AngularJS and Backbone take this to the point where the client layer can be considered an application in its own right, calling remote web services for tasks that it can't perform locally or to retrieve the data it needs to function (see figure 9.3). Mobile apps often use a similar architecture, with a significant amount of business logic residing within the client application.

In both these cases, the web services used by the rich client application form a clean API that can be effectively tested using non-UI testing, whereas the screen flow and behavioral logic of the UI can be effectively tested using web tests.

In the rest of the chapter, we'll look at a few strategies that can be applied in these different scenarios.

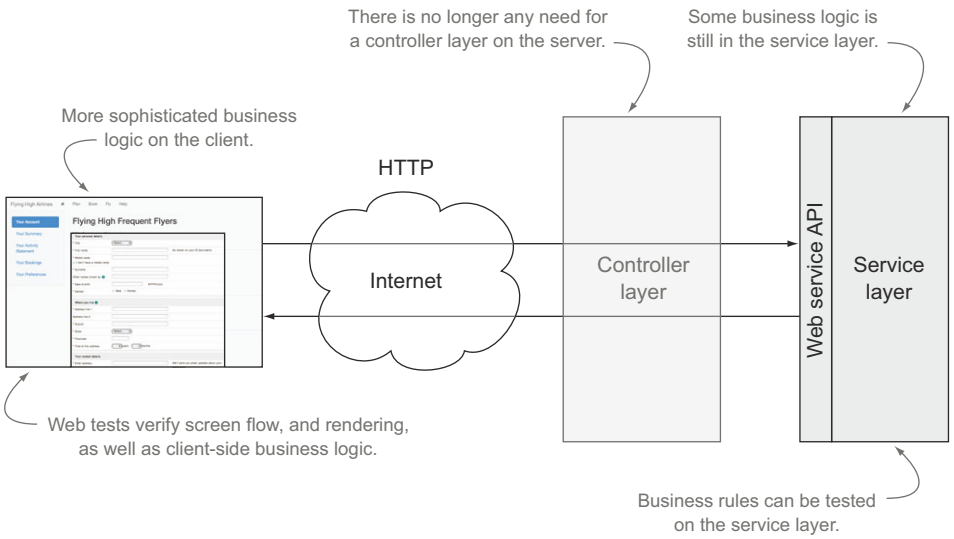


Figure 9.3 Modern single-page apps need more comprehensive tests of the business logic in both the UI and web service layer.